

Deep Learning Laboratory

Viva Preparation and Mathematical Guide

Steel Plate Fault Classification using PCA and
Backpropagation Neural Network

Siddharth Narayan Mishra
123CS0189

August 12, 2026

Contents

I	The 20% That Gives You 80%	5
1	How to Use This Guide	5
2	1. What Problem Are We Solving?	5
3	2. Why Do We Standardize?	6
4	3. What Is Covariance?	7
5	4. What Is PCA Actually Doing?	8
6	5. Why Eigenvalues and Eigenvectors?	8
7	6. Why Do We Sort Eigenvalues?	9
8	7. Explained Variance	9
9	8. Why Did We Choose 13 Components?	10
10	9. What Does “95% Variance” Actually Mean?	11
11	10. What Does the PCA Transformation Do?	11

12	11. What Is the Neural Network Doing?	12
13	12. Forward Propagation	13
14	13. Why Softmax?	13
15	14. ReLU vs Sigmoid	14
	15.1 ReLU	14
	15.2 Sigmoid	14
16	15. What Is the Loss Function?	15
17	16. What Is Backpropagation?	15
18	17. The Most Important Backpropagation Equation	16
19	18. Why Mini-Batches?	17
20	19. Why Do We Need a Test Set?	17
21	20. Accuracy vs Macro-F1	17
22	21. Understanding Our Results	18
23	22. Why Did PCA Model Perform Better?	19
24	23. The Parameter-Count Question	20
25	24. The Entire Experiment in One Mental Model	21
 II The Remaining 20%: Deeper Mathematics and Viva Questions		 22
26	25. Matrix Dimensions	22
27	26. Why Is the Covariance Matrix Symmetric?	22
28	27. Why Do PCA Components Become Uncorrelated?	23
29	28. Why Use eigh Instead of eig?	23
30	29. Why Is the Test PCA Transformation the Same as the Training	

PCA Transformation?	24
31 30. What Is Data Leakage?	24
32 31. What Is the Difference Between Parameters and Hyperparameters?	24
33 32. Why Random Seed 42?	25
34 33. Why Can Initial Weights Matter?	25
35 34. What Is an Epoch?	25
36 35. Why Does the Loss Curve Flatten?	26
37 36. What Does Convergence Mean?	26
38 37. Why Is Macro-F1 Better Than Accuracy Here?	26
39 38. Precision, Recall, and F1	27
40 39. What Does the Confusion Matrix Tell Us?	27
41 40. Understanding Our Confusion Matrices	28
42 41. Why Does PCA Reduce Neural-Network Parameters?	28
43 42. Does PCA Always Make Training Faster?	29
44 43. Why Did Configuration C Perform Better?	29
45 44. Why Can Sigmoid Cause Vanishing Gradients?	30
46 45. Why Is ReLU Popular?	30
47 46. Why Does the Output Layer Still Use Softmax?	31
48 47. What Is the Interface Supposed to Do?	31
49 48. Why Can't We Apply PCA Directly to a New Interface Input?	31
50 49. High-Probability Viva Questions	32
51 50. Five Equations You Should Be Able to Write From Memory	34

52 51. Five Numbers You Should Know	34
53 52. Final Mental Picture	35

Part I

The 20% That Gives You 80%

1 How to Use This Guide

The Pareto principle is useful for viva preparation:

You do not need to memorize every line of the implementation. You need to understand the small number of mathematical and conceptual ideas that explain almost everything the program does.

For this experiment, the most important chain is:

Standardization → Covariance → Eigenvalues/Eigenvectors → PCA → 95% Variance → Neural Network

If you understand this chain, most viva questions become straightforward.

The assignment specifically asks us to standardize the data, perform PCA using the training data, retain at least 95% variance, train a $27 \rightarrow 16 \rightarrow 8 \rightarrow 7$ neural network, test five activation configurations, compare the best PCA model with the original model, and build an interface.

2 1. What Problem Are We Solving?

We have a steel-plate dataset.

Each row represents one steel-plate sample.

Each sample has:

27

input features.

There are:

7

possible fault classes:

1. Pastry
2. Z_Scratch
3. K_Scratch

4. Stains
5. Dirtiness
6. Bumps
7. Other_Faults

Therefore, the machine-learning problem is:

27 numerical features $\rightarrow$ 7 possible classes

This is a **supervised multi-class classification problem**.

Why supervised?

Because we know the correct class of each training sample.

Why multi-class?

Because there are seven possible classes.

3 2. Why Do We Standardize?

Suppose one feature ranges from

0 to 1

while another ranges from

0 to 100000.

The second feature naturally has much larger numerical variance.

If PCA directly operates on such data, the large-scale feature can dominate the variance calculation.

Therefore we standardize.

The standardization formula is:

$$z = \frac{x - \mu}{\sigma}$$

where:

- x is the original value,
- μ is the training-set mean,
- σ is the training-set standard deviation,

- z is the standardized value.

After standardization:

$$\mu \approx 0, \quad \sigma \approx 1$$

for each training feature.

Viva answer:

We standardize because PCA is variance-based. Without standardization, features with larger numerical scales could dominate the covariance matrix and therefore dominate the principal components.

Very important: Why fit the scaler only on training data?

We calculate μ and σ using only training data.

Then we use those same values on the test data.

We must NOT calculate a separate mean and standard deviation from the test set.

Why?

Because the test set is supposed to represent unseen data.

Using test-set statistics during preprocessing is a form of information leakage.

4 3. What Is Covariance?

Covariance tells us how two variables change together.

For two variables X and Y :

$$Cov(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})$$

Interpretation:

- Positive covariance: they tend to increase together.
- Negative covariance: one tends to increase when the other decreases.
- Covariance near zero: little linear relationship.

For 27 features, instead of calculating one covariance value, we calculate all pairwise covariances.

This gives us a:

$$27 \times 27$$

covariance matrix.

5 4. What Is PCA Actually Doing?

This is the single most important concept in the experiment.

Suppose the original features are:

$$X_1, X_2, \dots, X_{27}.$$

Some of these features may contain redundant information.

PCA creates a new coordinate system:

$$PC_1, PC_2, \dots, PC_{27}.$$

The new variables are called principal components.

The first principal component captures the maximum possible variance.

The second captures the maximum remaining variance while being orthogonal to the first.

The third does the same after the first two, and so on.

Therefore:

$$PC_1 = \text{direction of maximum variance}$$

$$PC_2 = \text{next highest-variance orthogonal direction}$$

and so on.

6 5. Why Eigenvalues and Eigenvectors?

PCA obtains its principal components from the covariance matrix.

Let the covariance matrix be C .

We solve:

$$Cv = \lambda v$$

where:

- v is an eigenvector,

- λ is its eigenvalue.

The important interpretation is:

Remember

Eigenvector = direction of a principal component. Eigenvalue = amount of variance along that direction.

This is one of the most likely viva questions.

7 6. Why Do We Sort Eigenvalues?

Suppose the eigenvalues are:

$$8.25, \quad 3.43, \quad 2.68, \quad 2.31, \dots$$

The largest eigenvalue corresponds to the direction with the greatest variance.

Therefore we sort:

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_{27}.$$

Then:

$$PC_1$$

corresponds to λ_1 ,

$$PC_2$$

corresponds to λ_2 ,

and so on.

This gives us the principal components in order of importance.

8 7. Explained Variance

The total variance is proportional to the sum of all eigenvalues:

$$\sum_{j=1}^{27} \lambda_j.$$

The fraction of variance explained by component i is:

$$EV_i = \frac{\lambda_i}{\sum_{j=1}^{27} \lambda_j}$$

For example, our first principal component explained approximately:

30.54%

of the total variance.

The first two explained:

43.24%.

The first three explained:

53.15%.

And so on.

9 8. Why Did We Choose 13 Components?

The assignment requires at least:

95%

of the variance to be retained. :contentReference[oaicite:2]index=2

The cumulative variance obtained in our experiment was:

Components	Cumulative Variance
12	94.9502%
13	96.3781%

Since:

$94.9502\% < 95\%$

12 components are insufficient.

But:

$96.3781\% > 95\%$

so 13 components are sufficient.

Therefore:

$$d' = 13$$

and:

$$27 \rightarrow 13.$$

The dimensionality reduction is:

$$\frac{27 - 13}{27} \times 100 = 51.85\%$$

Viva answer:

We selected 13 components because 12 components retain only 94.9502% of the variance, which is below the required 95% threshold, while 13 components retain 96.3781%.

10 9. What Does “95% Variance” Actually Mean?

This is a very common trick viva question.

It does **not** mean:

95% classification accuracy.

It means that the selected principal components preserve at least 95% of the variance present in the original feature space.

PCA does not know the class labels.

Therefore:

95% variance retained does NOT imply 95% classification information retained.

A low-variance feature could still be extremely useful for distinguishing two fault classes.

PCA does not know this because PCA is unsupervised.

11 10. What Does the PCA Transformation Do?

After finding the eigenvectors, we project the standardized data onto the selected eigenvectors.

If W contains the first 13 eigenvectors:

$$W \in \mathbb{R}^{27 \times 13}$$

and the standardized input is:

$$X \in \mathbb{R}^{1550 \times 27},$$

then:

$$X_{\text{PCA}} = XW$$

and therefore:

$$(1550 \times 27)(27 \times 13) = 1550 \times 13.$$

Thus:

$$X_{\text{train,PCA}} \in \mathbb{R}^{1550 \times 13}$$

and similarly:

$$X_{\text{test,PCA}} \in \mathbb{R}^{391 \times 13}.$$

12 11. What Is the Neural Network Doing?

The baseline network is:

$$27 \rightarrow 16 \rightarrow 8 \rightarrow 7$$

This means:

- 27 input neurons,
- 16 neurons in hidden layer 1,
- 8 neurons in hidden layer 2,
- 7 output neurons.

The PCA network is:

$$13 \rightarrow 16 \rightarrow 8 \rightarrow 7.$$

The only difference is the input dimensionality.

13 12. Forward Propagation

For the first layer:

$$Z_1 = XW_1 + b_1$$

Then apply an activation function:

$$A_1 = f(Z_1).$$

Second layer:

$$Z_2 = A_1W_2 + b_2$$

$$A_2 = f(Z_2).$$

Output layer:

$$Z_3 = A_2W_3 + b_3.$$

Finally:

$$\hat{Y} = \textit{Softmax}(Z_3).$$

So the complete flow is:

$$X \rightarrow Z_1 \rightarrow A_1 \rightarrow Z_2 \rightarrow A_2 \rightarrow Z_3 \rightarrow \textit{Softmax}$$

14 13. Why Softmax?

We have seven classes.

The network produces seven output scores:

$$z_1, z_2, \dots, z_7.$$

Softmax converts these scores into probabilities:

$$P_i = \frac{e^{z_i}}{\sum_{j=1}^7 e^{z_j}}$$

Therefore:

$$0 \leq P_i \leq 1$$

and:

$$\sum_{i=1}^7 P_i = 1.$$

The predicted class is:

$$\hat{y} = \arg \max_i P_i$$

The largest probability is also what the assignment asks us to use as prediction confidence.
:contentReference[oaicite:3]index=3

15 14. ReLU vs Sigmoid

Two activation functions were used.

15.1 ReLU

$$ReLU(x) = \max(0, x)$$

Therefore:

$$ReLU(x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$

Its derivative is:

$$ReLU'(x) = \begin{cases} 0, & x < 0 \\ 1, & x > 0 \end{cases}$$

15.2 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Its derivative is:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

The maximum value of the sigmoid derivative is:

$$\frac{1}{4}.$$

This means sigmoid can produce relatively small gradients, especially when saturated.

16 15. What Is the Loss Function?

We use categorical cross-entropy.

For one sample:

$$L = - \sum_{i=1}^7 y_i \log(\hat{y}_i)$$

Because y is one-hot encoded, only the correct class contributes to the loss.

For example, if the correct class is class 3:

$$y = [0, 0, 1, 0, 0, 0, 0].$$

Then:

$$L = -\log(\hat{y}_3).$$

If the model assigns high probability to the correct class, loss is low.

If it assigns low probability to the correct class, loss is high.

17 16. What Is Backpropagation?

Backpropagation is the process of calculating how much each parameter contributed to the final error.

It uses the:

chain rule

from calculus.

The forward pass calculates:

$$X \rightarrow \hat{Y}.$$

The loss measures:

$$L(\hat{Y}, Y).$$

Backpropagation then calculates:

$$\frac{\partial L}{\partial W_3}, \quad \frac{\partial L}{\partial W_2}, \quad \frac{\partial L}{\partial W_1}.$$

The weights are updated using gradient descent:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

where η is the learning rate.

18 17. The Most Important Backpropagation Equation

For Softmax combined with categorical cross-entropy:

$$\frac{\partial L}{\partial Z_3} = \frac{\hat{Y} - Y}{m}$$

where m is the batch size.

This is an extremely useful viva equation.

From this:

$$dW_3 = A_2^T dZ_3$$

and:

$$db_3 = \sum dZ_3.$$

Then the gradient is propagated backward:

$$dA_2 = dZ_3 W_3^T.$$

For a ReLU layer:

$$dZ_2 = dA_2 \odot \text{ReLU}'(Z_2).$$

The same process continues toward the first layer.

19 18. Why Mini-Batches?

The batch size in the experiment is:

$$32.$$

Instead of calculating one update using all 1550 training samples, we calculate gradients using smaller batches.

Advantages include:

- less memory usage,
- more frequent parameter updates,
- computational efficiency.

With 1550 samples and batch size 32, there are approximately:

$$\left\lceil \frac{1550}{32} \right\rceil$$

updates per epoch.

20 19. Why Do We Need a Test Set?

The training set is used to learn parameters.

The test set is used to evaluate how well the learned model generalizes to unseen samples.

We obtained:

$$1550$$

training samples and:

$$391$$

test samples.

We must not use test data to train the network.

21 20. Accuracy vs Macro-F1

Accuracy is:

$$Accuracy = \frac{\text{correct predictions}}{\text{total predictions}}$$

However, our dataset is imbalanced.

For example, the class counts are:

Class	Samples
<i>Pastry</i>	158
<i>Z_Scratch</i>	190
<i>K_Scratch</i>	391
<i>Stains</i>	72
<i>Dirtiness</i>	55
<i>Bumps</i>	402
<i>Other_Faults</i>	673

Therefore, accuracy alone can be misleading.

Macro-F1 calculates F1 independently for each class:

$$Macro-F1 = \frac{1}{7} \sum_{i=1}^7 F1_i$$

where:

$$F1 = \frac{2PR}{P + R}.$$

Macro-F1 gives every class equal importance.

22 21. Understanding Our Results

The baseline model achieved:

$$Accuracy = 34.53\%$$

and:

$$Macro-F1 = 0.0733.$$

Its confusion matrix showed that it predicted every test sample as:

Other_Faults.

There were 135 test samples belonging to this class.

Therefore:

$$\frac{135}{391} \times 100 = 34.53\%.$$

So the model's accuracy was essentially obtained by always predicting the majority class.

This explains why the Macro-F1 was so low.

The model did not perform genuine seven-class classification well.

23 22. Why Did PCA Model Perform Better?

The best PCA model was:

$$\boxed{Sigmoid \rightarrow ReLU \rightarrow Softmax}$$

and achieved:

$$Accuracy = 49.10\%$$

and:

$$Macro-F1 = 0.2022.$$

Its confusion matrix shows that it correctly classified:

$$60$$

of the 79 K_Scratch test samples and:

$$132$$

of the 135 Other_Faults samples.

Thus:

$$60 + 132 = 192$$

correct predictions.

And:

$$\frac{192}{391} \times 100 \approx 49.10\%.$$

So PCA did not merely reduce dimensionality; in this particular experiment, the resulting representation also produced better classification performance.

24 23. The Parameter-Count Question

This is almost guaranteed to be asked.

For a fully connected layer:

$$\text{parameters} = (\text{input neurons} \times \text{output neurons}) + \text{output neurons}$$

The second term is the bias.

For the original network:

$$27 \rightarrow 16 :$$

$$27(16) + 16 = 448.$$

For:

$$16 \rightarrow 8 :$$

$$16(8) + 8 = 136.$$

For:

$$8 \rightarrow 7 :$$

$$8(7) + 7 = 63.$$

Total:

$$448 + 136 + 63 = 647$$

For the PCA model:

$$13 \rightarrow 16 :$$

$$13(16) + 16 = 224.$$

Therefore:

$$224 + 136 + 63 = 423.$$

PCA reduces parameters from:

647 $\rightarrow$ 423.

That is:

34.62%

fewer trainable parameters.

25 24. The Entire Experiment in One Mental Model

If the examiner asks:

“Explain your entire experiment.”

say:

We start with 27 steel-plate features and seven fault classes. We first perform a stratified train-test split and standardize the features using statistics learned only from the training set.

We then calculate the covariance matrix and perform eigendecomposition. The eigenvectors give the principal-component directions and the eigenvalues give the variance captured by those directions.

We select the minimum number of components required to retain at least 95% variance. In our experiment, 13 components retained 96.3781% variance, reducing dimensionality by 51.85%.

We then train a fully connected neural network using backpropagation. The baseline architecture is 27-16-8-7. For the PCA experiments the architecture becomes 13-16-8-7.

We test five activation configurations and evaluate them using accuracy, Macro-F1, training time, and loss curves. Configuration C, Sigmoid followed by ReLU, performed best with 49.10% accuracy and 0.2022 Macro-F1.

Finally, we compare the original and PCA models using performance, parameter count, training time, and confusion matrices.

If you can explain that naturally, you understand the experiment.

Part II

The Remaining 20%: Deeper Mathematics and Viva Questions

26 25. Matrix Dimensions

Knowing matrix dimensions is extremely useful in a viva.

For the original model:

$$X : 1550 \times 27$$

$$W_1 : 27 \times 16$$

Therefore:

$$XW_1 : 1550 \times 16.$$

Then:

$$W_2 : 16 \times 8$$

giving:

$$1550 \times 8.$$

Finally:

$$W_3 : 8 \times 7$$

giving:

$$1550 \times 7.$$

Therefore the output contains seven scores for every training sample.

27 26. Why Is the Covariance Matrix Symmetric?

The covariance between feature i and feature j is the same as the covariance between feature j and feature i :

$$\text{Cov}(X_i, X_j) = \text{Cov}(X_j, X_i).$$

Therefore:

$$C = C^T.$$

The covariance matrix is symmetric.

This is useful because symmetric matrices have real eigenvalues and orthogonal eigenvectors.

28 27. Why Do PCA Components Become Uncorrelated?

PCA uses orthogonal eigenvectors of the covariance matrix.

If:

$$v_i^T v_j = 0$$

for:

$$i \neq j,$$

then the corresponding principal directions are orthogonal.

When the data is projected onto these directions, the resulting principal components are uncorrelated.

29 28. Why Use `eigh` Instead of `eig`?

The covariance matrix is symmetric.

Therefore, an eigendecomposition routine designed for symmetric matrices is appropriate.

In NumPy:

```
np.linalg.eigh()
```

is designed for Hermitian/symmetric matrices.

This also provides numerically appropriate real-valued eigenvalues and eigenvectors for this problem.

30 29. Why Is the Test PCA Transformation the Same as the Training PCA Transformation?

Suppose training produces:

$$W_{PCA} \in \mathbb{R}^{27 \times 13}.$$

Training data is transformed as:

$$X_{train,PCA} = X_{train}W_{PCA}.$$

The test data must use the exact same W_{PCA} :

$$X_{test,PCA} = X_{test}W_{PCA}.$$

We do not calculate a new PCA on the test set.

Otherwise the model would be operating in a different feature space during testing.

31 30. What Is Data Leakage?

Data leakage occurs when information from outside the training data is used to train or preprocess the model.

Examples in this experiment would include:

- calculating the standardization statistics using the complete dataset,
- fitting PCA using the complete dataset,
- training the neural network using test samples.

The correct procedure is:

Train $\rightarrow$ Fit scaler $\rightarrow$ Fit PCA $\rightarrow$ Transform train/test $\rightarrow$ Train model $\rightarrow$ Evaluate test

32 31. What Is the Difference Between Parameters and Hyperparameters?

Parameters are learned during training:

$$W_1, W_2, W_3, b_1, b_2, b_3.$$

Hyperparameters are chosen before training.

Examples:

- learning rate = 0.01,
- batch size = 32,
- number of epochs = 100,
- hidden-layer sizes = 16 and 8.

33 32. Why Random Seed 42?

The assignment specifies a fixed random seed of 42 for reproducibility.

The network weights and biases are initialized randomly.

Without a fixed seed, running the same experiment multiple times could produce different initial weights and therefore different results.

34 33. Why Can Initial Weights Matter?

Suppose the initial weights are:

$$W^{(0)}.$$

Training begins from that point in the parameter space.

Different initializations can lead to different optimization trajectories and potentially different final solutions.

Therefore, fixing the random seed makes the experiment reproducible.

35 34. What Is an Epoch?

One epoch means that the model has processed the complete training dataset once.

With:

$$1550$$

training samples and:

$$32$$

samples per batch, approximately:

$$\left\lceil \frac{1550}{32} \right\rceil$$

mini-batches are processed per epoch.

36 35. Why Does the Loss Curve Flatten?

At the beginning of training, the network is far from a good solution, so the loss often decreases rapidly.

Later, the gradients become smaller or the optimizer approaches a region where further improvements are small.

Therefore the curve can plateau.

For Configurations A, B, D, and E, the loss dropped quickly and then stabilized around 1.67.

Configuration C continued decreasing more strongly toward the end of training.

37 36. What Does Convergence Mean?

Convergence means that the optimization process is approaching a stable region where the loss changes relatively little.

A curve that:

decreases rapidly → changes slowly

is showing convergence or approaching a plateau.

However:

Low training loss alone does not guarantee good test performance.

We care about generalization, which is why test accuracy and Macro-F1 are also measured.

38 37. Why Is Macro-F1 Better Than Accuracy Here?

Our classes are highly imbalanced.

The largest class contains:

673

samples, while Dirtiness contains only:

55.

A model that predicts mostly the largest class can obtain a seemingly reasonable accuracy while completely failing minority classes.

Macro-F1 prevents this from being hidden because each class contributes equally to the final metric.

39 38. Precision, Recall, and F1

For a particular class:

$$Precision = \frac{TP}{TP + FP}.$$

Recall is:

$$Recall = \frac{TP}{TP + FN}.$$

F1 combines them:

$$F1 = \frac{2PR}{P + R}$$

where P is precision and R is recall.

40 39. What Does the Confusion Matrix Tell Us?

Rows represent actual classes.

Columns represent predicted classes.

Therefore:

$$CM_{ij}$$

means:

number of samples whose actual class is i but were predicted as j

The diagonal:

$$CM_{ii}$$

contains correct predictions.

Off-diagonal elements represent misclassifications.

41 40. Understanding Our Confusion Matrices

The original model's matrix contains:

135

in the Other_Faults/Other_Faults position.

All other samples are also placed in the Other_Faults prediction column.

Therefore the model has essentially collapsed into majority-class prediction.

The PCA model is better because it has:

60

correct K_Scath predictions and:

132

correct Other_Faults predictions.

It still struggles with several minority classes.

Therefore the model is improved, but it is not a strong classifier overall.

42 41. Why Does PCA Reduce Neural-Network Parameters?

The first layer connects every input feature to every hidden neuron.

Originally:

27×16

weights.

After PCA:

13×16 .

The later layers remain unchanged.

Therefore:

$647 - 423 = 224$

parameters are removed.

The reduction comes entirely from reducing the input dimensionality.

43 42. Does PCA Always Make Training Faster?

No.

It reduces the number of input features and therefore the number of parameters.

However, measured training time depends on:

- matrix operations,
- activation functions,
- implementation overhead,
- hardware,
- memory operations,
- number of epochs.

Our results demonstrate this:

$$T_{\text{original}} = 0.4153s$$

while:

$$T_{\text{best PCA}} = 0.4911s.$$

Therefore, parameter reduction did not translate into lower measured training time in this small experiment.

44 43. Why Did Configuration C Perform Better?

Configuration C was:

$$\boxed{\text{Sigmoid} \rightarrow \text{ReLU}}$$

and obtained:

$$49.10\%$$

accuracy.

Its loss curve also continued decreasing more than the other configurations.

However, we should be careful about claiming a universal reason.

The correct experimental conclusion is:

For this dataset, initialization, architecture, optimization settings, and the selected activation ordering resulted in Configuration C achieving the best observed performance.

We cannot conclude that Sigmoid+ReLU is universally better than the other configurations.

45 44. Why Can Sigmoid Cause Vanishing Gradients?

The sigmoid derivative is:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)).$$

Its maximum is:

$$0.25.$$

For large positive or negative x :

$$\sigma(x) \approx 1$$

or:

$$\sigma(x) \approx 0.$$

Therefore:

$$\sigma'(x) \approx 0.$$

During backpropagation, gradients are repeatedly multiplied through layers. Repeated multiplication by small values can make gradients extremely small. This is called the vanishing-gradient problem.

46 45. Why Is ReLU Popular?

For positive values:

$$\text{ReLU}'(x) = 1.$$

Therefore gradients can pass through without being multiplied by a small derivative. ReLU is also computationally simple:

$$\max(0, x).$$

However, ReLU has its own issue: neurons can become permanently inactive when their inputs remain negative.

This is sometimes called the dying-ReLU problem.

47 46. Why Does the Output Layer Still Use Softmax?

The activation-function experiments change only the hidden-layer activations.

The output layer remains:

$$\text{Softmax}.$$

This is because the task remains seven-class classification.

The assignment explicitly specifies Softmax in the output layer for each configuration.
:contentReference[oaicite:4]index=4

48 47. What Is the Interface Supposed to Do?

The interface accepts:

27

original features.

It then performs:

27 features → standardization → PCA → 13 components → neural network → prediction

The assignment specifies that the fitted standardization and PCA transformation must be used, and that the maximum Softmax probability is the confidence.
:contentReference[oaicite:5]index=5

49 48. Why Can't We Apply PCA Directly to a New Interface Input?

Because PCA is not simply a formula that independently analyzes one sample.

It learned a transformation from the training data:

$$W_{PCA}.$$

For a new sample:

$$x \in \mathbb{R}^{27},$$

we first standardize it:

$$z = \frac{x - \mu_{train}}{\sigma_{train}}.$$

Then project:

$$x_{PCA} = zW_{PCA}.$$

The resulting 13-dimensional vector is then passed to the neural network.

50 49. High-Probability Viva Questions

1. What is PCA?

Answer: PCA is an unsupervised dimensionality-reduction technique that transforms correlated variables into orthogonal principal components ordered by the amount of variance they explain.

2. Why standardize before PCA?

Answer: Because PCA is variance-based and features with larger scales could otherwise dominate the covariance matrix.

3. What is an eigenvector in PCA?

Answer: It gives the direction of a principal component.

4. What is an eigenvalue?

Answer: It represents the amount of variance captured along the corresponding eigenvector.

5. Why did we select 13 components?

Answer: Twelve components explain only 94.9502%, while thirteen explain 96.3781%, so 13 is the minimum satisfying the 95% requirement.

6. Does 95% explained variance mean 95% classification accuracy?

Answer: No. PCA preserves variance, not class-discriminative information.

7. Why use Softmax?

Answer: To convert the seven output scores into probabilities summing to one.

8. Why categorical cross-entropy?
Answer: Because this is a seven-class classification problem.
9. What is backpropagation?
Answer: It calculates gradients of the loss with respect to network parameters using the chain rule.
10. What is gradient descent?
Answer: It updates parameters in the negative gradient direction to reduce the loss.
11. Why use Macro-F1?
Answer: Because the dataset is imbalanced and Macro-F1 gives equal importance to every class.
12. How many parameters does the original model have?
Answer: 647.
13. How many parameters does the PCA model have?
Answer: 423.
14. Which configuration performed best?
Answer: Configuration C, Sigmoid + ReLU, with 49.10% accuracy and 0.2022 Macro-F1.
15. What did the baseline confusion matrix show?
Answer: The baseline predicted every test sample as Other_Faults.
16. Why did that produce 34.53% accuracy?
Answer: Because 135 of the 391 test samples were Other_Faults, and $135/391 = 34.53\%$.
17. What does Macro-F1 prevent us from hiding?
Answer: Poor performance on minority classes.
18. Why is the PCA model not necessarily faster?
Answer: Fewer parameters do not guarantee lower wall-clock time because runtime also depends on implementation and computational overhead.
19. Why must PCA be fitted only on training data?
Answer: To prevent test-set information leakage.
20. What does the interface confidence represent?
Answer: The maximum Softmax probability among the seven classes.

51 50. Five Equations You Should Be Able to Write From Memory

If you are short on time, memorize these five equations and understand what every symbol means.

1. Standardization

$$z = \frac{x - \mu}{\sigma}$$

2. PCA Eigenvalue Equation

$$Cv = \lambda v$$

3. Explained Variance

$$EV_i = \frac{\lambda_i}{\sum_j \lambda_j}$$

4. Softmax

$$P_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

5. Gradient Descent

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

52 51. Five Numbers You Should Know

For this particular experiment, remember these:

Quantity	Value
Original dimensions	27
PCA dimensions	13
Variance retained	96.3781%
Dimensionality reduction	51.85%
Original parameters	647
PCA parameters	423
Best configuration	Sigmoid + ReLU
Best accuracy	49.10%
Best Macro-F1	0.2022

53 52. Final Mental Picture

The entire experiment can be remembered using four questions.

Question 1: Why PCA?

27 → 13

Reduce dimensionality while retaining at least 95% variance.

Question 2: How does PCA work?

Covariance → *Eigenvalues/Eigenvectors* → *PrincipalComponents*

Question 3: How does the neural network learn?

Forward Pass → *Loss* → *Backpropagation* → *Gradient Descent*

Question 4: What did our experiment show?

13 PCA features → 423 parameters → 49.10% accuracy → 0.2022 Macro-F1

The most important overall conclusion is:

PCA reduced the dimensionality from 27 to 13 and reduced neural-network parameters from 647 to 423. In our experiment, the best PCA-based model also improved classification performance from 34.53% to 49.10% accuracy and from 0.0733 to 0.2022 Macro-F1, although it did not reduce measured training time.